

Machine Learning

Contents

1. Introduction	4
What Does It Mean to Learn?	4
Traditional Programming and Machine Learning	4
The Parts of a Learning Problem	4
When Machine Learning Is Useful	5
The Three Main Learning Paradigms	5
Supervised Learning	5
Unsupervised Learning	5
Reinforcement Learning	5
Worked Example: A Spam Filter	6
Model-Based and Instance-Based Learning	6
Designing a Learning System	6
Typical Failure Modes	6
Questions to Ask Before Using Machine Learning	7
Takeaway	7
2. Mathematical Preliminaries / Machine Learning Workflow	7
The End-to-End Flow	7
Data Objects and Attributes	8
Data Quality	9
IQR Flagging	9
Three-Sigma Flagging	9
Sampling and Partitioning	9
Learned parameters and hyperparameters	10
Feature selection and feature extraction	10
Leave-one-out and k-fold validation	10
Scaling and Encoding	10
Feature Engineering	11
Worked Example: Predicting Loan Repayment	11
Common Mistakes	11
From Offline Score to Real Use	12
Takeaway	12
Linear Models for Regression	12
From inputs to a prediction	12
Two ways to fit the model	13

Hypothesis and parameter count	13
Direct solution	13
Gradient descent	14
Measuring prediction quality	14
Worked Example: Estimating Delivery Time	15
Linear does not mean straight-only	15
Bias, variance, and regularization	16
Practical checklist	16
Common Misunderstandings	16
Residual Diagnostics	17
Linear Models for Classification	17
Score and decision boundary	17
Discriminant functions	18
Generative and discriminative approaches	18
Decisions need thresholds and costs	18
A compact example	19
Score, label, and confidence are different	19
Worked Example: Quality Inspection	19
What the Geometry Tells Us	19
Practical checklist	20
Takeaway	20
Logistic Regression	20
Probability from a linear score	20
Why not use linear regression for probability?	21
Cross-entropy loss	21
Why confident errors cost more	22
Compact example	22
Regularization and preprocessing	23
From Probability to a Decision	23
Common Mistakes	23
Practical checklist	23
Takeaway	23

1. Introduction

Machine learning matters when a useful rule is easier to learn from examples than to write by hand. A spam filter, for example, can learn which combinations of words and sender patterns predict spam instead of depending on a permanently growing list of human-written rules.

[Detailed](#) · [External Sources](#)

What Does It Mean to Learn?

The lecture used Tom Mitchell's task–performance–experience framing. A program learns when its performance improves on a task because of experience.

- Task, T: what the system must do.
- Performance measure, P: how success is measured.
- Experience, E: the data or interaction from which it learns.

For handwriting recognition:

- T is assigning the correct character to an image.
- P could be the percentage of unseen images classified correctly.
- E is a collection of images paired with human-provided labels.

This framing prevents a vague statement such as “build an intelligent system.” It forces us to identify an observable task, a measurable result, and the evidence available for learning.

Traditional Programming and Machine Learning

In traditional programming, a developer supplies data and explicit rules; the computer returns an output. In supervised machine learning, the developer supplies examples of inputs and desired outputs; a learning algorithm estimates a model that can later produce outputs for new inputs.

The learned model is not magic and does not “understand” in the human sense. It is a mathematical mapping whose parameters were adjusted to perform well on observed examples.

The Parts of a Learning Problem

A supervised learning problem becomes easier to reason about when its parts are separated:

- An example is one case, such as one email or one house sale.
- Features are the input measurements available when a prediction is made.
- A target is the desired answer used during training.
- A model maps features to a prediction.
- Parameters are values learned from training data.
- A loss function measures how undesirable a prediction is during training.
- A metric summarizes performance in terms meaningful to evaluation or the real decision.

Loss and metric need not be identical. A model may optimize a smooth mathematical loss while the organization reports recall, error cost, or another operational metric.

When Machine Learning Is Useful

The lectures emphasized several useful situations:

- The rule is hard to express but examples are available, as in handwriting or speech recognition.
- Patterns change over time, as in spam or fraud detection.
- The model must be personalized, as in recommendations.
- The volume or dimensionality of data makes manual rule writing impractical.
- Decisions improve through repeated interaction, as in game-playing or robotic control.

Machine learning is not automatically the right choice. A stable payroll calculation, for example, is clearer, cheaper, and safer as an explicit program.

The Three Main Learning Paradigms

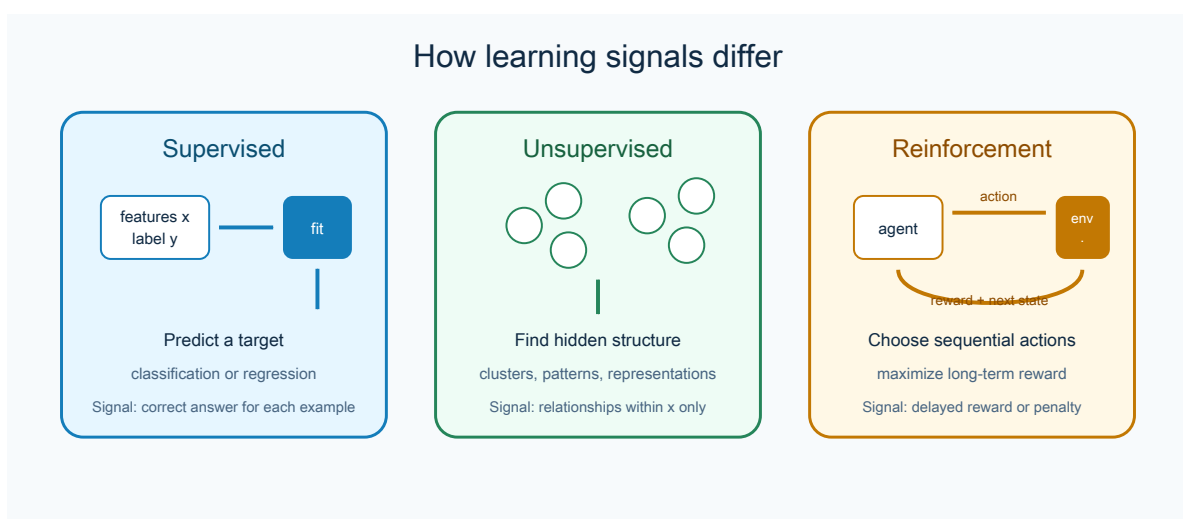


Figure 1. The kind of feedback available determines the learning paradigm.

Supervised Learning

Supervised learning uses labelled examples (x, y) . The input x contains features; the target y is the desired answer.

- Classification: y is a category, such as spam/not-spam or job-offered/not-offered.
- Regression: y is a numerical value, such as house price or humidity.

The goal is not merely to reproduce the training examples. It is to generalize to previously unseen cases.

Unsupervised Learning

Unsupervised learning receives inputs x without target labels. It searches for useful structure, such as customer groups or lower-dimensional representations. In clustering, a useful result usually places similar observations together and separates dissimilar groups.

Reinforcement Learning

An agent acts in an environment and receives rewards or penalties. The challenge is sequential: an action can affect later states and later rewards. The objective is therefore long-term reward, not simply the correctness of one isolated prediction.

The lecture also mentioned semi-supervised learning, where a small labelled set is combined with a larger unlabelled set.

Worked Example: A Spam Filter

Suppose the task is to classify incoming email as spam or legitimate mail.

- 1 Each historical email is an example.
- 2 Features may describe words, sender history, links, and formatting.
- 3 The target is the human-reviewed class: spam or legitimate.
- 4 The model learns parameters from older labelled emails.
- 5 Performance is measured on later emails that were not used for fitting.

Accuracy alone may be insufficient. Marking an important legitimate message as spam can be more costly than allowing one advertisement into the inbox. The performance measure should therefore reflect both kinds of error and the users affected by them.

The deployment environment can also change. Spammers adapt their wording, so a once-successful model may degrade. Monitoring and carefully collected new labels become part of the learning system.

Model-Based and Instance-Based Learning

A model-based learner fits a general rule or parameterized model from the training data, then applies that learned model to new inputs. Linear regression is a familiar example: training estimates coefficients, and prediction uses the resulting equation.

An instance-based learner retains training examples and answers a new query by comparing it with relevant stored cases. K-nearest neighbours can classify a query from the labels of nearby training examples. Its prediction cost and sensitivity to feature scaling are part of the design trade-off.

Do not confuse k-nearest neighbours with k-means. K-nearest neighbours is commonly used for supervised classification or regression; k-means is an unsupervised clustering method that represents groups by centroids. The shared letter k only names a chosen count.

Designing a Learning System

A practical learning system requires more than selecting an algorithm.

- 1 Define the task and the real-world decision it supports.
- 2 Choose a performance measure that reflects the cost of mistakes.
- 3 Decide what experience is available and whether it represents future cases.
- 4 Choose the target and features without leaking future information.
- 5 Select a model family and training procedure.
- 6 Evaluate on data that was not used to fit or tune the model.
- 7 Monitor the deployed system because data and behaviour can change.

The workflow and data-quality stages are developed in [Know More](#).

Typical Failure Modes

- Unrepresentative experience: training examples differ from deployment cases.
- Poor target definition: the label is convenient but does not measure the real goal.
- Data leakage: training uses information that would be unavailable at prediction time.
- Wrong metric: a high aggregate score hides expensive errors or weak performance on an important group.
- Overfitting: the model learns accidental details of the training sample.
- Feedback effects: model decisions change the data that will be collected later.

Questions to Ask Before Using Machine Learning

- 1 Is there a clear prediction or decision to support?
- 2 Can the correct outcome be measured reliably?
- 3 Are representative examples available at the time the model must learn?
- 4 Would a simpler rule-based solution be safer or easier to maintain?
- 5 What happens when the model is wrong, and who bears that cost?
- 6 How will performance be monitored after deployment?

Takeaway

Machine learning is a disciplined way to improve performance from experience. Start with **T**, **P**, and **E**; then choose the learning paradigm from the feedback available. The quality of the task definition and evidence usually matters more than the novelty of the algorithm.

2. Mathematical Preliminaries / Machine Learning Workflow

A machine-learning model is only one stage in a larger evidence pipeline. Reliable work starts by understanding the data, separating evaluation data early, and ensuring every learned preprocessing step is fitted without access to the final test set.

[Detailed](#) · [External Sources](#)

The End-to-End Flow

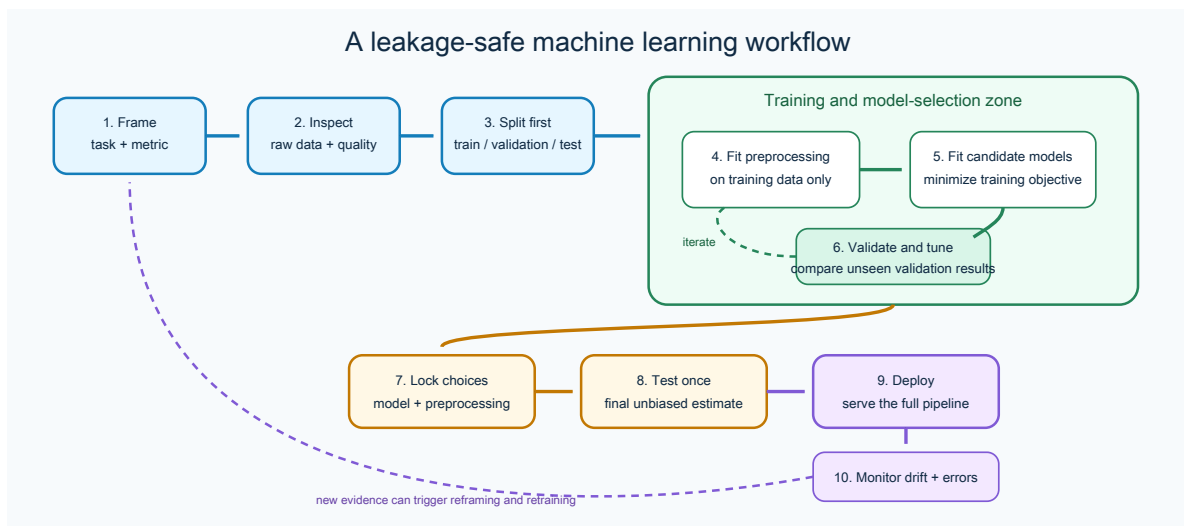


Figure 1. Split before fitting transformations; reserve the test set for the final estimate.

The lecture organized practice around data representation, parameter optimization, and model evaluation. A robust workflow expands that idea:

- 1 Frame the task, target, metric, and prediction time.
- 2 Collect and inspect relevant data.
- 3 Split into training, validation, and test roles.
- 4 Fit cleaning, imputation, encoding, scaling, and feature selection on training data.
- 5 Fit candidate models on transformed training data.
- 6 Use validation data or cross-validation to tune choices.
- 7 Lock the complete pipeline.
- 8 Evaluate once on the held-out test set.
- 9 Deploy the same transformation-and-model pipeline.
- 10 Monitor data quality, drift, errors, and downstream effects.

Data Objects and Attributes

An object is one record, instance, case, or sample. An attribute is a measured property, field, variable, or feature.

Attribute scale	What comparisons are meaningful?	Example
Nominal	equality or difference	city, colour, product code
Ordinal	equality and order	low/medium/high, rating level
Interval	order and differences	Celsius temperature, calendar year
Ratio	order, differences, and ratios	mass, duration, Kelvin temperature

Attributes may also be discrete (countable values) or continuous (values on a numeric continuum). These distinctions affect encoding, summaries, distance calculations, and valid interpretations.

Data Quality

The lectures emphasized that poor data can deny credit to qualified applicants or approve risky applicants. Common problems include:

- measurement noise;
- invalid or inconsistent values;
- missing values;
- duplicates created when sources are merged;
- rare observations or outliers;
- unrepresentative or imbalanced samples;
- misleading, irrelevant, or redundant features.

Cleaning is not equivalent to deleting every unusual row. First determine whether an unusual value is an error, a valid rare case, or the event the analysis is supposed to detect.

IQR Flagging

Let Q_1 and Q_3 be the first and third quartiles. The interquartile range is:

$$IQR = Q_3 - Q_1$$

Tukey's common inner fences are:

$$\text{lower fence} = Q_1 - 1.5 \times IQR$$

$$\text{upper fence} = Q_3 + 1.5 \times IQR$$

Values beyond these fences are candidates for investigation.

Three-Sigma Flagging

For data adequately described by a normal distribution, approximately 99.73% of the population lies within $\mu \pm 3\sigma$, where μ is the population mean and σ is the population standard deviation.

For an exactly normal distribution, the proportion is about 99.73%. This rule is unreliable for strongly skewed, heavy-tailed, or multimodal data, so first inspect the distribution and the domain meaning of the flagged value. [External Sources](#)

Sampling and Partitioning

A sample supports generalization only when it represents the cases on which the model will operate. Small samples create sampling noise; flawed collection can create sampling bias even when the sample is large.

- Simple random sampling gives eligible observations a defined random chance of selection.
- Stratified sampling preserves important group proportions, such as class labels.
- Cluster sampling samples groups rather than independent individuals.

- Under-sampling removes majority examples; over-sampling adds or repeats minority examples. Both alter the training distribution and must be applied inside the training process only.

The training set estimates model parameters. The validation set or cross-validation process selects models and hyperparameters. The test set estimates performance after choices are locked.

Learned parameters and hyperparameters

Learned parameters are values fitted from training examples, such as regression weights and an intercept. Hyperparameters are choices made before or around fitting, such as the learning rate, batch size, number of folds, or regularization strength. Validation data help compare hyperparameter choices; the final test set must remain untouched until the choices are fixed.

Feature selection and feature extraction

The 6 September lecture distinguished two preprocessing decisions. Feature selection keeps the most useful existing columns and discards irrelevant or redundant ones. Feature extraction combines or transforms existing columns into a new, more useful feature—for example, combining height and weight into a body-mass-index feature. These decisions belong inside the training pipeline so that information from the held-out validation or test data does not leak into the fitted model.

Leave-one-out and k-fold validation

In leave-one-out validation, one example is held out for validation and the model is trained on all remaining examples; the process is repeated so every example is held out once. In k-fold cross-validation, the training data are split into k folds. Each fold is used once as validation while the other k – 1 folds are used for training, and the validation scores are averaged. For k = 5, each example participates in one validation fold and four training folds.

Scaling and Encoding

Min–max scaling to [0, 1] is:

$$x' = \frac{x - \min_{\text{train}}}{\max_{\text{train}} - \min_{\text{train}}}$$

Standardization is:

$$z = \frac{x - \mu_{\text{train}}}{s_{\text{train}}}$$

The subscript train is essential: fit the minimum, maximum, mean, standard deviation, imputer, encoder, and feature selector on training data only. Apply those learned transformations unchanged to validation, test, and production data.

Both displayed formulas require a nonzero training range or standard deviation. A constant feature contains no variation for the model to learn from, so remove it or follow the library's documented

constant-feature behavior rather than dividing by zero.

Standardization does not require normally distributed data. It is useful when feature scale affects optimization, distance, or regularization, although its mean and standard deviation can be strongly influenced by outliers.

Categorical variables may be represented with one-hot encoding or another explicit encoding. Label encoding can accidentally introduce a numerical order when categories are nominal.

Feature Engineering

- Extraction: derive a lower-dimensional representation.
- Selection: retain useful existing features and remove irrelevant or redundant ones.
- Construction: combine raw features using domain knowledge or interactions.
- Transformation: change representation, such as logarithms, scaling, or binning.

Continuous features do not universally need discretization. Decision trees and k-nearest-neighbour methods can use continuous inputs directly, and Gaussian Naive Bayes models continuous features. Binning is a deliberate modelling choice: it can simplify a pattern, but it discards within-bin differences and creates artificial boundaries.

Worked Example: Predicting Loan Repayment

Suppose a lender wants to predict whether an applicant will repay a loan. At application time it knows income, requested amount, employment length, and previous repayment history.

- 1 Define the prediction moment. Information created after approval, such as the eventual number of missed payments, cannot be a feature.
- 2 Split by time. Train on older applications and test on later applications to imitate future use.
- 3 Fit preprocessing on training data. Estimate the income median, category vocabulary, and scaling parameters from the training period only.
- 4 Handle imbalance. If defaults are rare, accuracy alone can be misleading; inspect recall, precision, and the cost of missed defaults.
- 5 Tune on validation data. Select the model and decision threshold without examining final test outcomes.
- 6 Evaluate once. Apply the locked pipeline to the test period and report both predictive performance and group-level error rates.

This example shows why leakage is not merely a software mistake. A feature may exist in the database today but still be unavailable at the moment when the real decision must be made.

Common Mistakes

- Scaling the complete dataset before splitting, which lets test-set statistics influence training.
- Treating every missing value as zero, even when zero has a real domain meaning.
- Removing every outlier without checking whether it represents fraud, failure, or another important rare event.
- Repeatedly checking the test score while tuning; the test set then becomes part of model selection.

- Deploying a model without the exact preprocessing pipeline used during training.

From Offline Score to Real Use

An offline test score estimates behaviour only under conditions represented by the held-out data. Deployment can introduce new effects:

- Data drift: input distributions change, such as customer behaviour shifting after a policy change.
- Concept drift: the relationship between inputs and the target changes, such as fraud strategies adapting.
- Training-serving skew: production features are computed differently from training features.
- Feedback loops: model decisions influence which future examples and labels become observable.

Monitoring should therefore include input quality, missing-value rates, prediction distributions, delayed outcome metrics, and important subgroup performance. A model is not finished merely because one test score is acceptable.

Takeaway

Split early, fit preprocessing only on training data, preserve the same pipeline for every later input, and treat unusual data as evidence to investigate. A trustworthy workflow protects the evaluation boundary from the first transformation onward.

Linear Models for Regression

Regression predicts a continuous quantity, such as price, demand, temperature, or travel time. A linear model is a useful starting point because its predictions are easy to compute and its parameters have a clear interpretation.

[Detailed](#) · [External Sources](#)

From inputs to a prediction

For one feature, simple linear regression uses

$$\hat{y} = w_0 + w_1 x$$

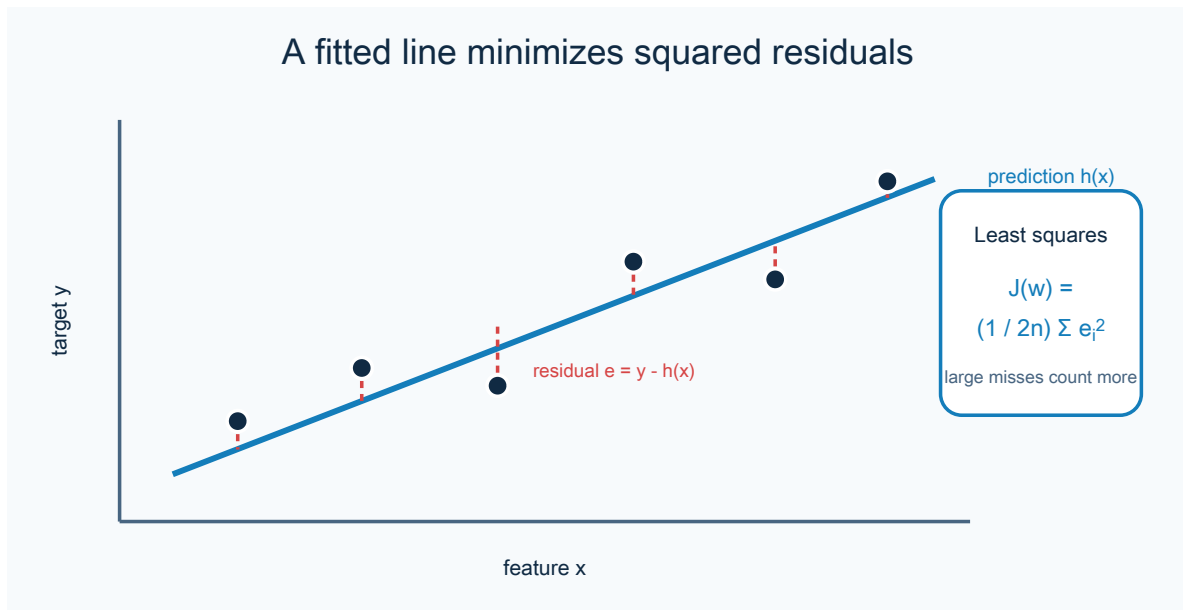
Here, x is the input, $\hat{y}$ is the prediction, w_0 is the intercept, and w_1 is the slope. With d features, the model becomes

$$\hat{y} = w_0 + \sum_{j=1}^d w_j x_j = \mathbf{x}_{\text{aug}}^T \mathbf{w}$$

where the augmented vector includes a leading 1 for the intercept:

$$X_{\text{aug}} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

For an entire dataset, the prediction vector is $\hat{y} = Xw$.



The residual for observation i is $e_i = \hat{y}_i - y_i$. Least-squares regression chooses parameters that minimize

$$J(w) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Squaring prevents positive and negative errors from cancelling and penalizes larger errors more strongly.

Two ways to fit the model

Hypothesis and parameter count

The model can be viewed as a hypothesis: a rule that maps the feature matrix to predictions. With d input features and an intercept, the parameter vector has $d + 1$ entries. Training learns these entries from the training data; the learning rate, batch size, and choice of fitting method are not learned coefficients.

Direct solution

Setting the gradient of the least-squares objective to zero gives the normal equations:

$$X^T X w = X^T y$$

If $X^T X$ is invertible, this gives

$$w = (X^T X)^{-1} X^T y$$

This inverse formula requires independent columns and acceptable numerical conditioning. In practice, use a least-squares solver or matrix factorization; such methods also handle rank-deficient data more safely than explicitly forming an inverse.

Gradient descent

Gradient descent starts with an initial parameter vector and repeatedly updates every parameter:

$$w_j \leftarrow w_j - \alpha \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

where $\alpha > 0$ is the learning rate and $x_{i0} = 1$. A very small learning rate makes slow progress; a very large one can overshoot or diverge. Feature scaling often makes one learning rate work more evenly across parameters.

- Batch gradient descent: uses all training examples per update.
- Stochastic gradient descent: uses one example per update.
- Mini-batch gradient descent: uses a small subset and is the usual practical compromise.

In batch descent, one parameter update uses all training examples. In mini-batch descent, one update uses the selected batch; in stochastic descent, one update uses one training example. The supplied batch size therefore changes how many updates occur and how noisy each gradient estimate is.

The direct method is attractive for modest problems. Iterative methods become useful when the dataset or feature set is large. [Know More](#)

Measuring prediction quality

Mean squared error (MSE):

$$MSE = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

Lower is better, but its unit is squared.

Root mean squared error (RMSE):

$$\text{RMSE} = \sqrt{\text{MSE}}$$

RMSE is expressed in the target's original unit.

R-squared:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

R-squared compares the model with predicting the sample mean. It can be negative on evaluation data. The displayed ratio is undefined when every evaluated target equals the same value, so software may apply a documented finite-value convention.

Always compute evaluation metrics on data not used to fit the parameters.

Worked Example: Estimating Delivery Time

Assume a fitted one-feature model predicts delivery time from distance:

$$\hat{y} = 12 + 3.5x$$

Here x is distance in kilometres, 12 minutes is the estimated preparation and pickup time, and 3.5 minutes per kilometre is the estimated distance effect. For a 6 km order,

$$\hat{y} = 12 + 3.5(6) = 33 \text{ minutes}$$

If the actual delivery takes 38 minutes, the residual under the convention $e = \hat{y} - y$ is

$$e = 33 - 38 = -5 \text{ minutes}$$

The negative residual means the model underpredicted by five minutes. One case does not prove that the slope is wrong: traffic, restaurant delay, and weather are omitted factors. A residual plot across many deliveries can reveal whether errors remain random or systematically change with distance.

Linear does not mean straight-only

A model can be nonlinear in the original input but linear in its parameters. For example,

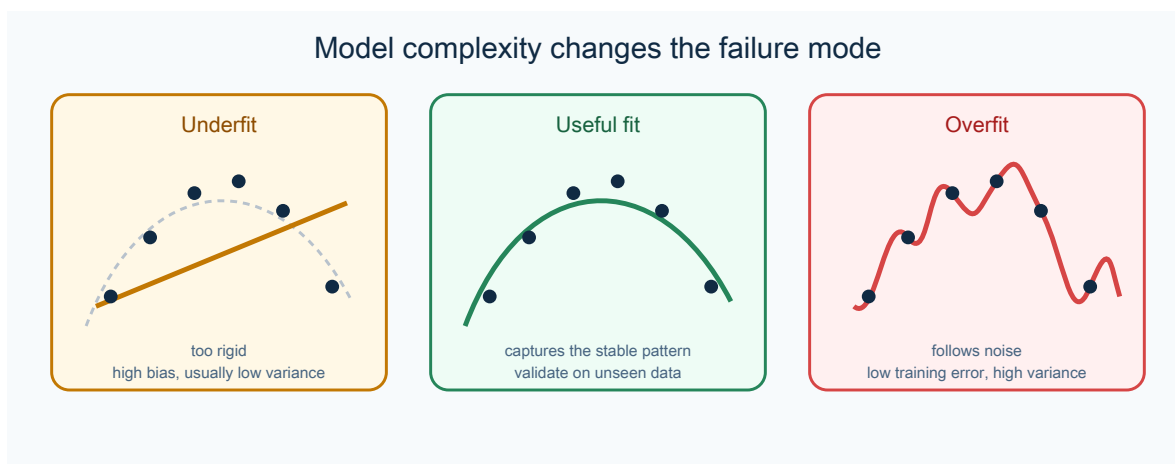
$$\hat{y} = w_0 + w_1x + w_2x^2 = \mathbf{w}^T \boldsymbol{\varphi}(x)$$

with basis vector

$$\boldsymbol{\varphi}(x) = \begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}$$

Increasing polynomial degree increases flexibility but can also increase sensitivity to noise.

Bias, variance, and regularization



- High bias: the model is too restrictive and underfits both training and evaluation data.
- High variance: the model follows training details too closely and performs poorly on new data.

Regularization discourages excessive parameter magnitude. Ridge uses an L2 penalty:

$$J(w) + \lambda \sum_{j=1}^d w_j^2$$

Lasso uses an L1 penalty:

$$J(w) + \lambda \sum_{j=1}^d |w_j|$$

Elastic net combines L1 and L2 penalties.

The intercept is commonly excluded from the penalty. Larger λ increases shrinkage and may increase bias while reducing variance. Select it using validation data rather than the test set.

Practical checklist

- 1 Separate training, validation, and test roles before fitting.
- 2 Inspect target units, missing values, and suspicious observations.
- 3 Scale features when using gradient-based fitting or regularization.
- 4 Compare training and validation errors to diagnose fit.
- 5 Report a metric whose units and limitations are clear.

Common Misunderstandings

- A large coefficient is not automatically the most important feature; coefficient size depends on units and feature scale.
- A high training R^2 does not guarantee useful future predictions.

- Linear regression describes association under its model assumptions; it does not establish causation.
- Extrapolating far outside the observed feature range can produce implausible predictions.
- Squared loss gives unusual observations strong influence, so inspect residuals and data quality.

Residual Diagnostics

A residual is more useful as part of a pattern than as an isolated number. After fitting, plot residuals against predictions or important features.

- A curved pattern suggests that the supplied linear features miss nonlinearity.
- A widening band suggests non-constant error spread.
- Separate clusters may indicate an omitted group or context variable.
- One extreme residual may be a data error, a rare valid case, or evidence that the model is unsuitable for that region.

Least squares does not require every feature to be normally distributed. Distributional assumptions become important when constructing confidence intervals or hypothesis tests. For prediction, the immediate checks are whether the relationship is represented adequately, errors are acceptable on unseen data, and extrapolation is controlled.

Linear Models for Classification

Classification predicts a category rather than a continuous number. A linear classifier summarizes the input with a weighted score and separates classes using a line, plane, or higher-dimensional hyperplane.

[Detailed](#) · [External Sources](#)

Score and decision boundary

For a feature vector $\mathbf{x}$, define the score

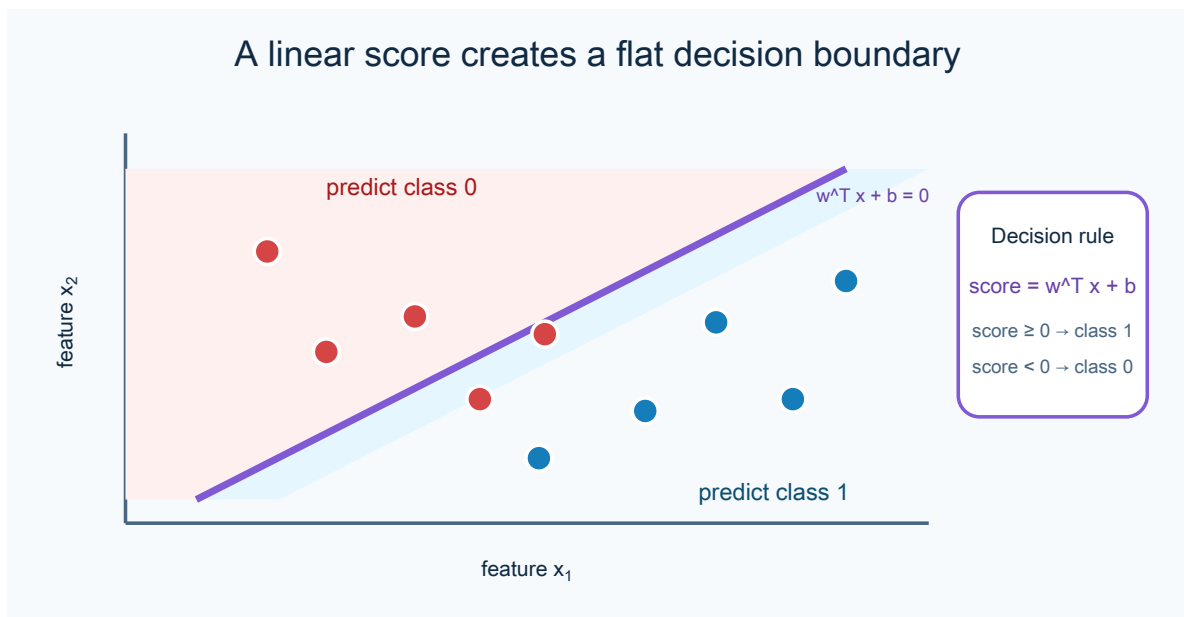
$$a(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{w}$ contains feature weights and b is the intercept. A basic binary rule is

$$\hat{y} = \begin{cases} 1, & \text{if } a(\mathbf{x}) \geq 0 \\ 0, & \text{if } a(\mathbf{x}) < 0 \end{cases}$$

The decision boundary is the set of inputs satisfying

$$\mathbf{w}^T \mathbf{x} + b = 0$$



The vector w is perpendicular to the boundary. Its direction points toward increasing scores, while b shifts the boundary without rotating it.

Discriminant functions

A discriminant function assigns a score to each class. In multiclass notation, choose

$$\hat{y} = \arg \max_k g_k(x)$$

For a linear discriminant, $g_k(x) = w_k^T x + b_k$. The boundary between classes j and k occurs where $g_j(x) = g_k(x)$.

The lectures used this idea to connect geometric separation with probabilistic classifiers. Logistic regression, covered next, transforms a linear score into a binary class probability. [Know More](#)

Generative and discriminative approaches

- A generative classifier models how each class could generate inputs, commonly through $p(x | y)$ and $p(y)$, then applies Bayes' rule.
- A discriminative classifier models $p(y | x)$ directly or learns a decision boundary without modeling the full input distribution.

Naive Bayes is a standard generative example; logistic regression is a discriminative example.

Neither family is universally better. Performance depends on how well the assumptions fit the data, the amount and dimensionality of the data, optimization, regularization, and the evaluation setting.

Decisions need thresholds and costs

A probability estimate does not determine a decision until a threshold or loss rule is chosen. With equal costs, a binary classifier often uses threshold 0.5. When a false negative is more costly than a false positive, a lower threshold may be appropriate.

Changing the threshold changes predicted labels but does not retrain the underlying probability model. Evaluate the resulting trade-off on validation data and reserve the test set for final assessment.

A compact example

Let $a(x) = 2x_1 - x_2 - 1$.

- For $x = (2, 1)$, $a = 2$, so the rule predicts class 1.
- For $x = (0, 2)$, $a = -3$, so the rule predicts class 0.
- Points satisfying $2x_1 - x_2 - 1 = 0$ lie on the boundary.

The score's sign controls the class. Its magnitude is not itself a geometric distance because rescaling both w and b changes the score without moving the boundary. The perpendicular distance is

$$\frac{|w^T x + b|}{\|w\|_2}$$

A raw score is also not automatically a calibrated probability.

Score, label, and confidence are different

The model first produces a score. A decision rule converts that score into a label. Neither step alone proves that the numerical score is a reliable probability.

For example, scores 0.2 and 20 lie on the same side of a zero threshold, so both receive class 1. The second point is farther from the boundary under a fixed parameter scale, but describing it as “100 times more probable” would be unjustified. Probability interpretation requires a probabilistic model such as logistic regression and may still require calibration checks.

Worked Example: Quality Inspection

A factory classifies a component using thickness x_1 and surface roughness x_2 :

$$a(x) = 1.5x_1 - 2x_2 - 0.4$$

For a component with $x_1 = 1.2$ and $x_2 = 0.3$,

$$a(x) = 1.5(1.2) - 2(0.3) - 0.4 = 0.8$$

The nonnegative score places it on the accepted side of the boundary. However, acceptance should still reflect the cost of errors. If shipping a defective component is dangerous, the operating threshold may be moved to demand a larger positive score, increasing caution at the cost of rejecting more acceptable parts.

What the Geometry Tells Us

- Each weight controls how the score changes when that feature increases while others stay fixed.
- The boundary is linear in the supplied feature space; useful nonlinear boundaries require transformed features or another model family.
- Features with incompatible units should be scaled before comparing coefficient magnitudes or applying distance-based interpretations.

- A separating boundary can fit training data perfectly yet generalize poorly when it relies on noise or unstable features.

Practical checklist

- 1 Define the positive class and error costs before choosing a metric.
- 2 Fit preprocessing only on training data.
- 3 Inspect class balance and use more than accuracy when imbalance matters.
- 4 Tune thresholds on validation data, not on the test set.
- 5 Check whether a linear boundary is plausible for the available features.

Takeaway

A linear classifier converts features into a weighted score and uses a boundary to separate decisions. Geometry explains which side of the boundary a point occupies; thresholds and costs determine the operational label; a separate probabilistic interpretation is required before calling the output a probability.

Logistic Regression

Logistic regression is a binary classification model that turns a linear score into a probability. It keeps the interpretability of a linear boundary while ensuring predictions lie between zero and one.

[Detailed](#) · [External Sources](#)

Probability from a linear score

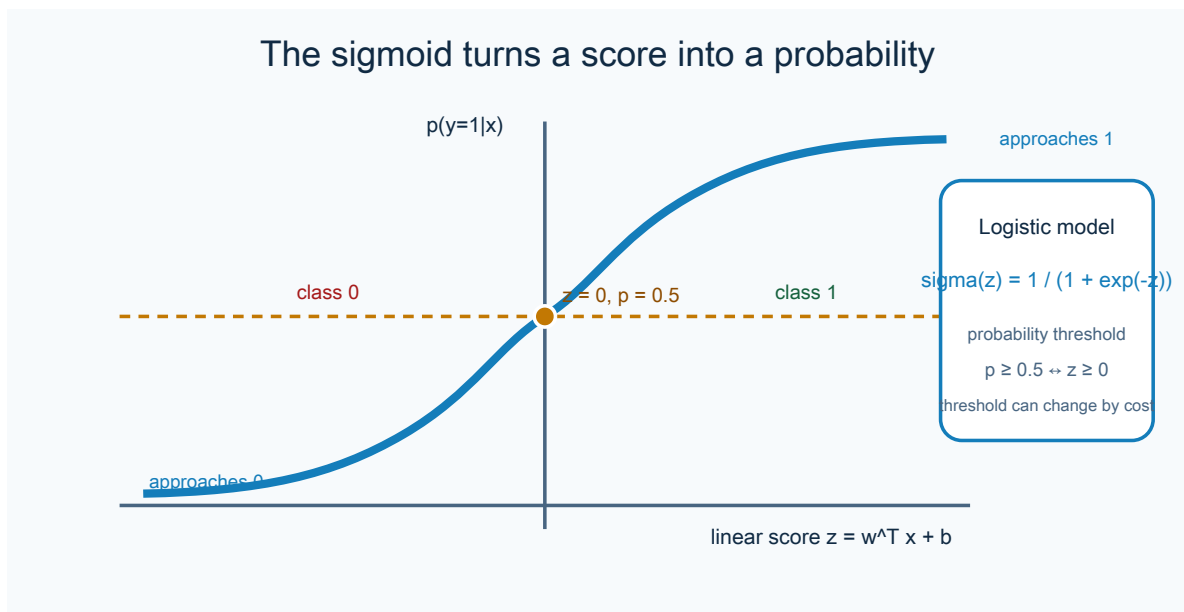
First compute

$$z = w^T x + b$$

Then apply the sigmoid function:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The model interprets p as $P(y = 1 | x)$. As z increases, p approaches 1; as z decreases, p approaches 0.



With a probability threshold of 0.5, predict class 1 when $p \geq 0.5$. Because $\sigma(0) = 0.5$, the corresponding boundary is $w^T x + b = 0$.

Why not use linear regression for probability?

An unconstrained linear prediction can be below 0 or above 1. Clipping those values loses a coherent probability model and does not produce the logistic loss used for Bernoulli outcomes. Logistic regression instead models the log-odds linearly:

$$\ln \left(\frac{p}{1-p} \right) = w^T x + b$$

A one-unit increase in feature x_j , with other features fixed, multiplies the odds by $\exp(w_j)$.

Cross-entropy loss

For binary label y in $\{0, 1\}$, the loss for one example is

$$L(y, p) = -[y \ln(p) + (1 - y) \ln(1 - p)]$$

The mean training objective is

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n L(y_i, p_i)$$

This loss strongly penalizes confident wrong predictions. For the unregularized binary model, its gradients are

$$\partial J / \partial \mathbf{w} = \frac{1}{n} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

and

$$\partial J / \partial b = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)$$

Gradient descent applies $\mathbf{w} \leftarrow \mathbf{w} - \alpha \partial J / \partial \mathbf{w}$ and $b \leftarrow b - \alpha \partial J / \partial b$.

Why confident errors cost more

For a positive example ($y = 1$), the one-example loss becomes $-\ln(p)$.

Predicted p	Interpretation	Loss $-\ln(p)$
0.90	confident and correct	about 0.105
0.60	correct but uncertain	about 0.511
0.10	confident and wrong	about 2.303

The loss is small when the correct class receives high probability and grows rapidly as the model becomes confidently wrong. This gives optimization a strong signal to correct severe mistakes.

Compact example

Suppose normalized academic score and aptitude score are both 1, with $b = -1$, $w_1 = 0.8$, and $w_2 = 0.4$. Then

$$z = -1 + 0.8(1) + 0.4(1) = 0.2$$

and

$$p = \sigma(0.2) \approx 0.550$$

At threshold 0.5, the model predicts the positive class. The probability is only modestly above the threshold, so it should not be described as high confidence.

The odds corresponding to this probability are

$$\frac{p}{1 - p} \approx \frac{0.550}{0.450} \approx 1.22$$

So the model assigns about 1.22 positive-outcome odds for every 1 negative-outcome odds. Odds are not the same as probability: odds of 3 correspond to probability $3 / (1 + 3) = 0.75$.

Regularization and preprocessing

Logistic regression often includes an L2, L1, or elastic-net penalty to control coefficient magnitude. Scale features when their units differ substantially, particularly when regularization or gradient-based solvers are used. Fit every preprocessing step on training data only.

Solver and penalty combinations vary by software implementation. Check the installed library's documentation rather than assuming every solver supports every penalty.

From Probability to a Decision

The default threshold 0.5 is only a convention. Consider medical screening: lowering the threshold can detect more patients who may have the condition, but it also sends more healthy patients for additional testing. Choose the threshold using validation data and explicit error costs, then report it with the final metrics.

Probability quality and ranking quality are different. A model can rank risky cases well while producing probabilities that are systematically too high or too low. When decisions depend on the numerical probability itself, inspect calibration as well as classification metrics.

Common Mistakes

- Reading w_j as a direct probability change; it is a change in log-odds, and e^{w_j} is an odds multiplier.
- Comparing coefficient magnitudes when features use very different scales.
- Calling a thresholded label “the probability.” The probability is p ; the label follows from a decision rule.
- Selecting the decision threshold on the final test set.
- Assuming predicted probabilities are calibrated without checking them on held-out data.

Practical checklist

- 1 Define which outcome is encoded as 1.
- 2 Inspect class balance and choose suitable metrics.
- 3 Scale and transform using training data only.
- 4 Tune regularization and the decision threshold on validation data.
- 5 Evaluate once on untouched test data and report the threshold used.

Takeaway

Logistic regression uses a linear score for ranking and a sigmoid for probability. Cross-entropy trains the probabilities, regularization controls coefficient growth, and a separately chosen threshold converts probability into action. Always interpret coefficients through log-odds and evaluate the final threshold in the context of error costs.